TrustNav — oral examination: speaker notes

Group 2, MSc Cybersecurity and Data Science, ESAIP Angers · Friday 13 November 2026, 09:35, room C201 · Tutor: Sofiane Hamrioui

Running order (about 17 minutes at a calm pace, then questions)

Slide	Speaker	Time	Message in one line
1 Title	Karan	1 min	One question: can a fleet believe its satellite position?
2 Problem	Sudheer	1.5 min	Real incidents; nothing today scores, explains and acts per vehicle.
3 System	Karan	1.5 min	A software layer beside the receiver: scores, explains, responds.
4 Witnesses	Karan	2 min	Rules, anomaly model, OSNMA; fused by noisy-OR, not averaged.
5 Response	Karan	1.5 min	Accept, hold, release; nothing downstream consumes a lie.
6 Evaluation	Niranjan	2 min	Disjoint routes; 99.3 % precision, 96.1 % recall, 0.44 % false alarms, with intervals.
7 Timelines	Niranjan	1.5 min	7/7 episodes, median 1 s, 8 km reduced to 29 m, no blank screen.
8 Real attacks	Niranjan	2.5 min	The first version missed a real spoofer for 21 min 49 s; a 16th rule catches it in the same second.
9 Demo	Karan + Pavan	2.5 min	Replay the teleport, the map, the phone as a receiver.
10 Claims	Pavan	1.5 min	Limits, ethics, next steps; one closing sentence.

Before the room

Open **trustnav.vercel.app** in a browser tab before the talk; load Replay → *Teleport spoof* and pause it before the attack (press **P** for presentation mode if the room screen is small).

Useful during questions, each one screen: Results → *Real attacks* (the captured phone, the four detectors compared, real jamming), Results → *Each attack* (what an ordinary screen would have shown against what TrustNav hands on), Results → *Real drive*, Method → *Try it* (two sliders, average against noisy-OR) and Method → *The 16 rules*.

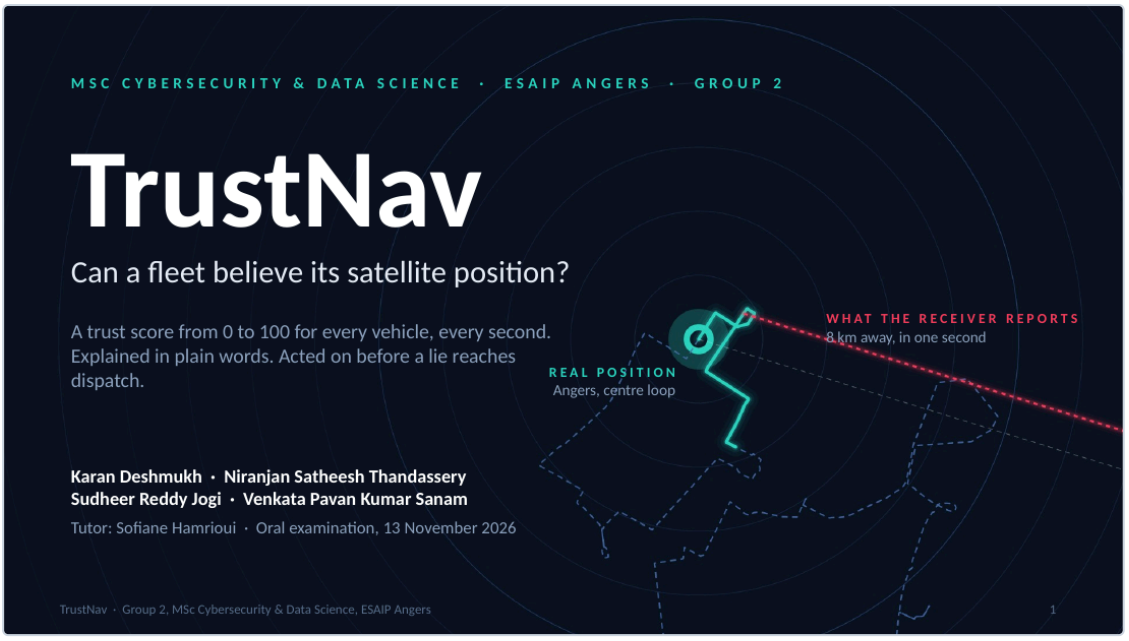
Phone: open **trustnav.vercel.app/phone**, token pasted, outdoors or near a window for a fix; start the laptop engine (`python -m trustnav_engine.phone`) with the token in `engine/.env.local`. If no fix indoors, show the attack switch on the recorded replay instead.

Fallbacks if the network fails: the PDF of this deck, the paper's Fig. 2 and Fig. 3, and the screenshots on slides 5 and 9. Say so in one sentence and move on.

If the jury gives you less time: slides 1 to 5 in one sentence each (the last column above), then spend the time on slides 6 and 8 and on the demo. Slide 8 is the answer to “is this only simulated?”.

Numbers to keep in mind: 36 features, 16 rules, 10 held-out scenarios, 3 800 observations (1 550 attacked, 2 250 clean), 42 tests, 109 end-to-end checks. Real attacks: a 26.5-minute capture, 21 min 49 s before the first version alerted, 909 of 909 captured fixes refused with the 16th rule, 15 of 20 jamming exposures alerted. Clean real drive: 27.6 minutes, no false alarm from the rules.

1 TrustNav: can a fleet believe its satellite position?



Speaker: Karan (1 min)

Good morning. We are Group 2 and this is TrustNav. One question sums up the project: can a fleet believe the satellite position its vehicles report?

Every dispatch screen, geofence and flight controller today assumes the answer is yes. TrustNav is a software layer that checks that assumption once a second, for every vehicle and drone, gives each position a trust score from 0 to 100, explains the score in plain words, and acts on it when the position cannot be trusted.

In the next quarter of an hour: why the problem is real, how TrustNav decides, how we evaluated it, what happened when we replayed recordings of real attacks, and a live demonstration. We will keep the demo short and leave time for your questions.

2 Everything that moves trusts GNSS blindly



Speaker: Sudheer (1.5 min)

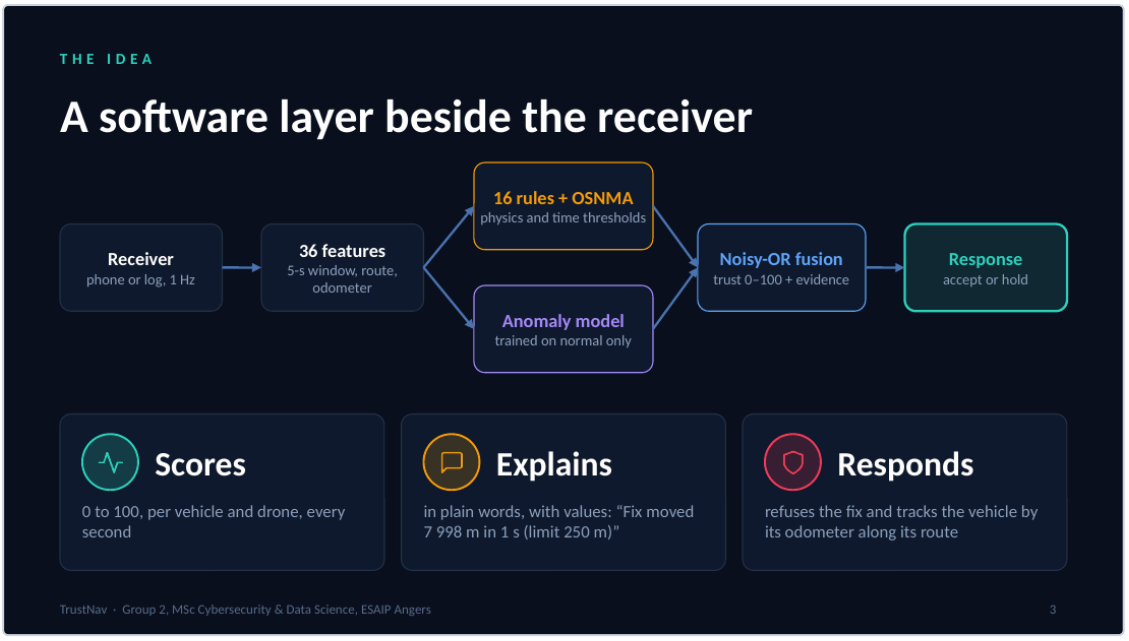
Why does this matter? GNSS signals arrive from 20 000 km away and are extremely weak, so a small transmitter nearby can drown them, which is jamming, or imitate them and feed a believable false position, which is spoofing.

This is not theoretical and not far away. In April 2017 a jammer left running in a car parked at Nantes airport delayed several flights; the owner was fined two thousand euros. In July 2023 a truck driver near Lyon used a jammer to escape his company's tracking and disrupted the airport until the ANFR traced him. And since 2022 interference has been a daily event for aviation around the Baltic and conflict zones, about fifteen hundred flights a day spoofed in the summer of 2024.

What exists today does not close the gap. Fleet telematics simply displays the position it is given; at best it relays a jamming flag. Receiver manufacturers like u-blox or Septentrio do detect inside the receiver, but they output a state code with no explanation, and at Jammertest 2025 both the jamming and the spoofing flags tripped during jamming alone. Fixed probes and anti-jamming antennas protect a site, a port or an airport; they do not travel with a van.

So the gap is this: nobody tells the operator, per vehicle and per second, whether the position is true, and nobody acts on it. That is what TrustNav does.

3 A software layer beside the receiver



Speaker: Karan (1.5 min)

TrustNav sits beside the receiver and works only on what any receiver already reports: position, speed, satellites, signal strength, dilution of precision, clock, and where available the Galileo authentication status and the vehicle's own odometer speed. The source can be a real receiver, a phone, or a recorded log, once per second.

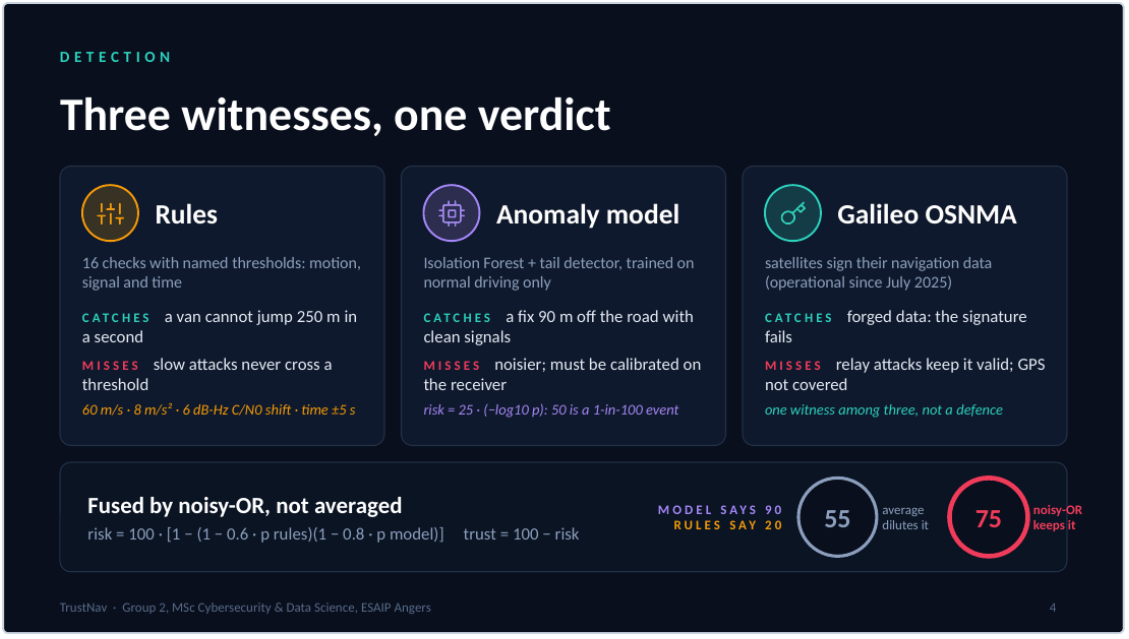
From each second and the five before it, we compute 36 features: how far the fix moved, derived versus reported speed, distance to the planned route, changes in signal strength, satellite count, clock, and the mismatch between GNSS speed and odometer speed. Absent inputs are flagged explicitly, never mistaken for zero.

Two independent witnesses then judge the features: sixteen rules with named thresholds, and an anomaly model trained on normal driving only. Their risks are fused by noisy-OR into one trust score between 0 and 100, with the evidence attached.

The score drives a response: an accepted fix goes downstream unchanged; a refused fix is replaced by an estimate. Everything is also sent to the console and the incident log.

Three promises follow: TrustNav scores, every second; it explains, in plain words with the actual values; and it responds, so a lie never reaches dispatch, geofencing or flight control.

4 Three witnesses, one verdict



Speaker: Karan (2 min)

Let me open the three witnesses.

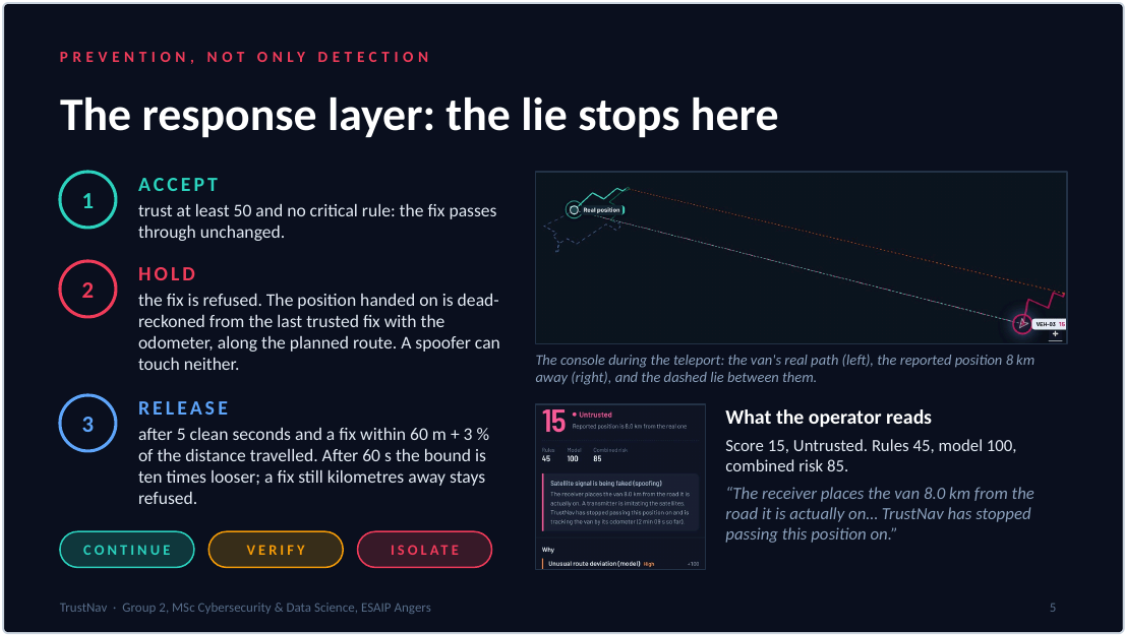
The rules are sixteen deterministic checks against named thresholds. A road vehicle cannot exceed 60 metres per second, accelerate at more than 8 metres per second squared, or jump 250 metres in one second. On the signal side, a 6 dB-Hz shift in carrier-to-noise ratio, a PDOP above 5, a clock-drift change above 60 nanoseconds per second. The sixteenth is about time: the receiver's clock may not move more than five seconds against the clock of the device it is plugged into. It came out of a real attack, and slide 8 tells that story. Three conditions are critical and pin the risk at 70 or more: a jump of four times the limit, a failed OSNMA signature, and a time jump. The rules are almost silent on normal data, and that is also their weakness: a slow attack never crosses a threshold.

The anomaly model covers that blind spot. It is an Isolation Forest with 300 trees plus a per-feature tail detector, and it is trained on normal operation only, never on an attack, so it cannot memorise the simulator's attacks. Its raw score is converted to a calibrated tail probability on normal runs it has not seen, and the risk is 25 times minus log10 of that probability. A risk of 50 therefore means a one-in-a-hundred event under normal driving. The tail detector also names the odd input: a fix 90 metres off the road while the signals are clean.

Galileo OSNMA is the third witness: the satellites sign their navigation data, operational since July 2025. A failed signature means forged data and is critical. A valid signature is not proof of safety: a relay attack re-broadcasts genuine signals, and GPS is not covered at all.

The witnesses are combined by noisy-OR, not averaged. If the model says 90 and the rules say 20, an average would give 55, barely over the alert line, and with silent rules a model alarm of 90 would average to 45 and never alert; noisy-OR gives 75 and 72. The two weights were chosen by grid search on validation routes only.

5 The response layer: the lie stops here



The console during the teleport: the van's real path (left), the reported position 8 km away (right), and the dashed lie between them.

Speaker: Karan (1.5 min)

Detection alone leaves the operator looking at a lie on the screen. The response layer turns the score into an action, the way an intrusion prevention system acts on what an intrusion detection system finds.

A fix is accepted when trust is at least 50 and no critical rule fired. Otherwise it is held: the fix is refused, and the position handed to downstream software is dead-reckoned from the last trusted fix using the odometer speed, advanced along the planned route. A spoofer can fake satellites; it cannot touch the odometer or the route.

The hold is released only after five clean seconds and a returning fix that rejoins the estimate within 60 metres plus 3 percent of the distance travelled, so a spoofer cannot end the hold by simply looking clean while still far away. After sixty clean seconds the tolerance is loosened ten times, because an odometer estimate drifts too, but a fix still kilometres away stays refused.

Every second also carries an action for the operator: continue, verify by other means, or isolate the device when trust is below 25 or a critical rule fired.

On the right is what the operator sees: the score, the band, the three risks and the verdict in plain words, and on the map the real position, the protected position on the road, and the reported lie eight kilometres away.

6 Tested on routes it has never seen



Speaker: Niranjan (2 min)

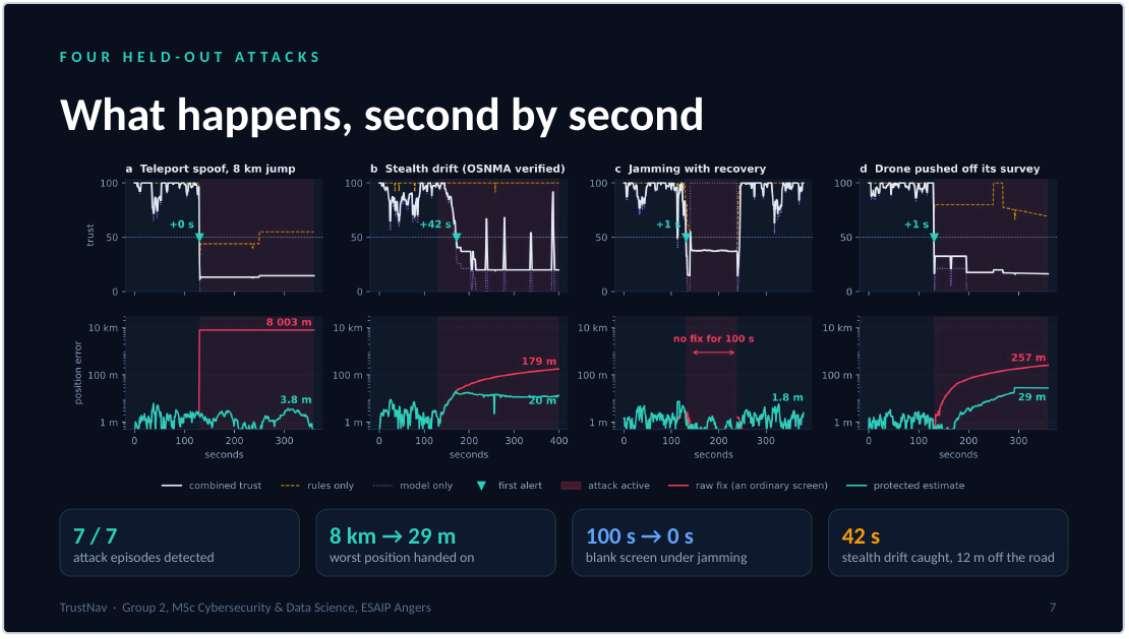
How did we test it? No public dataset gives labelled per-second receiver observables for fleets with routes and odometers, and transmitting attacks is illegal, so a deterministic, seeded simulator produces what a receiver would report, over real Angers street geometry.

The important discipline is that every route has exactly one role. Five road loops and two drone areas train the model, with normal runs only. Two loops and one drone area are validation, and the only thing chosen on them is the pair of fusion weights. Two loops and one drone area are held out and produce every number we report: ten scenarios, 3 800 one-second observations, 1 550 of them under attack, seven attack types. Rule thresholds come from physics, not from data, and an alert is a combined risk of 50 or more.

The chart compares four ways to decide. The baseline is the classic signal check, carrier-to-noise below 35 dB-Hz or fewer than five satellites: perfect precision but 9 percent recall, because it only sees jamming. The rules alone never raise a false alarm but miss slow attacks, 28 percent recall. The model alone reaches 90 percent recall but is noisier, 1.33 percent false alarms. Combined, each covers the other's blind spot: 99.3 percent precision, 96.1 percent recall, F1 0.977, 0.44 percent false alarms.

Because seconds within a run are correlated, we also bootstrapped over scenarios: the 95 percent intervals are 97.9 to 100 for precision and 89.4 to 99.6 for recall. At the event level, all seven attack episodes are detected, the median delay is one second, and no false incident is ever opened.

7 What happens, second by second



Speaker: Niranjan (1.5 min)

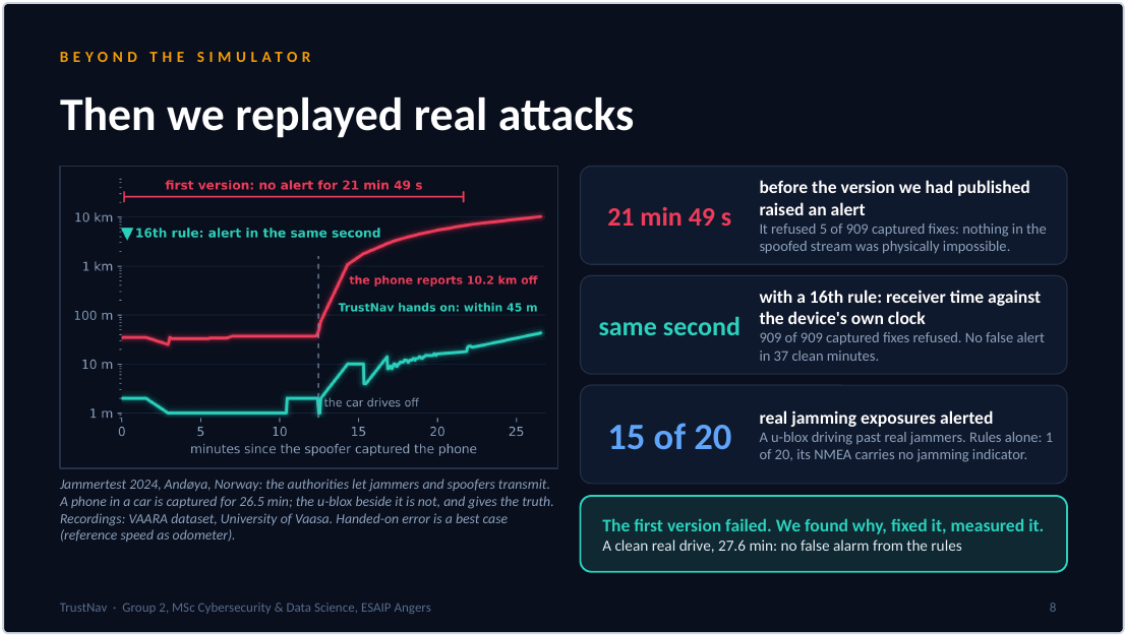
This figure shows four held-out scenarios second by second. The top row is trust: the blue line is the combined detector, the dashed and dotted lines are the rules and the model alone, the grey band is the attack, and the marker is the first alert with its delay.

Panel a, the teleport: the fix jumps 8 kilometres and is caught in the same second. Panel b is the one we are proudest of, the stealth drift: a relay-style attack that keeps the signals clean and Galileo authentication valid, moving the fix at 0.7 metres per second. No rule ever fires. The model catches it after 42 seconds, when the fix is about 12 metres off the road, where a fixed 150-metre route rule would need three and a half minutes. Panel c is jamming with recovery, caught after one second. Panel d is a drone pushed off its survey line.

The bottom row is what the response layer prevents. The orange line is the position error an ordinary screen would show; the green line is the error of the estimate TrustNav hands downstream, on a log scale. The worst reported error across the spoofing scenarios is 8 003 metres; the worst protected error is 29 metres. During the 100-second jamming outage an ordinary screen is blank; the estimate stays within 2 metres.

So: seven out of seven episodes, a median delay of one second, 8 kilometres reduced to 29 metres, and no blank screen.

8 Then we replayed real attacks



Speaker: Niranjan (2.5 min)

Everything so far came from one simulator. So we looked for real attacks that somebody else had recorded legally. Every September at Jammertest, in Norway, the authorities let jammers and spoofers transmit so that industry can test against them. The University of Vaasa drove a car through the 2024 edition with a u-blox receiver and two phones and published the recordings, the VAARA dataset. We replayed them, unchanged, through the same detector.

The chart is one phone during a spoofing test. The spoofer captured it for 26 and a half minutes: it set the phone's clock back an hour and its position 34 metres aside. The car was parked, so the lie looked like a parked car. Then the car drove off and the phone stayed where the spoofer had put it, 10.2 kilometres from the truth at the end. The u-blox beside it was not captured, so it gives the true position.

Here is the result we did not want: the version we had published missed it. First alert after 21 minutes 49 seconds; 5 of 909 captured fixes refused. Nothing in the spoofed stream was physically impossible.

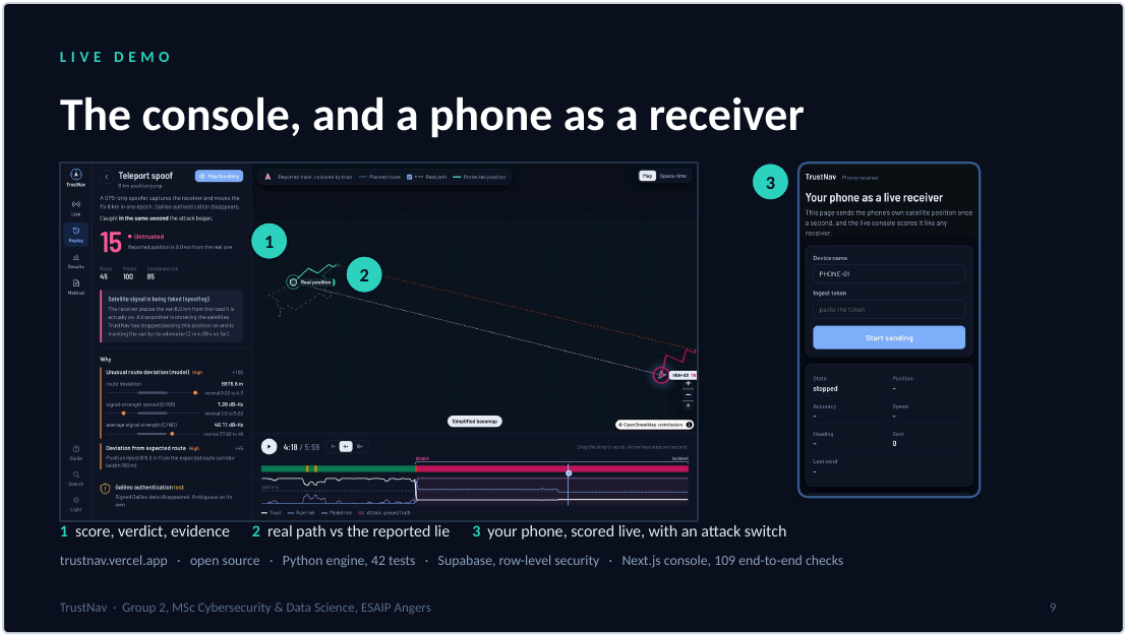
What the spoofer could not hide was time. The phone's log stamps every sentence with the phone's own clock, and the receiver's time had jumped against it. So we added a sixteenth rule: receiver time may not move more than five seconds against the clock of the device it is plugged into. Now the alert comes in the same second, all 909 captured fixes are refused, and there is no false alert in the 37 clean minutes before. The position handed on stays within 45 metres, a best case, because the reference speed stands in for the odometer.

Jamming, same campaign: the u-blox drove past real jammers twenty times. The rules alone alerted once, because its NMEA stream carries no jamming indicator; with the model re-calibrated on the clean 38 minutes before the first test, 15 of 20 are alerted, about ten seconds in.

And on a clean real drive published by Google, the rules raised no false alarm in 27.6 minutes.

So, honestly: the first version failed on a real attack. We found why, fixed it with one explainable rule, and measured it again. It is all on the Results page of the site, under Real attacks, and one command reproduces it.

9 Live demo: the console, and a phone as a receiver



Speaker: Karan, with Pavan driving the console (2.5 min)

Now the live system at trustnav.vercel.app. The opening screen plays the attack by itself, the van on its real route, the reported position thrown 8 kilometres away and the trust score falling to 12: leave it on screen while you introduce the demo. Then three things, briefly.

One: Replay, teleport spoof. Press play and step to the attack. The score falls from the nineties to about 12, the band turns Untrusted, and the verdict panel says in plain words that a transmitter is imitating the satellites and that the receiver places the van 8 kilometres from the road it is on. Below it, the evidence lines with the values and their normal ranges: this is what makes the decision checkable by an operator who is not a GNSS expert.

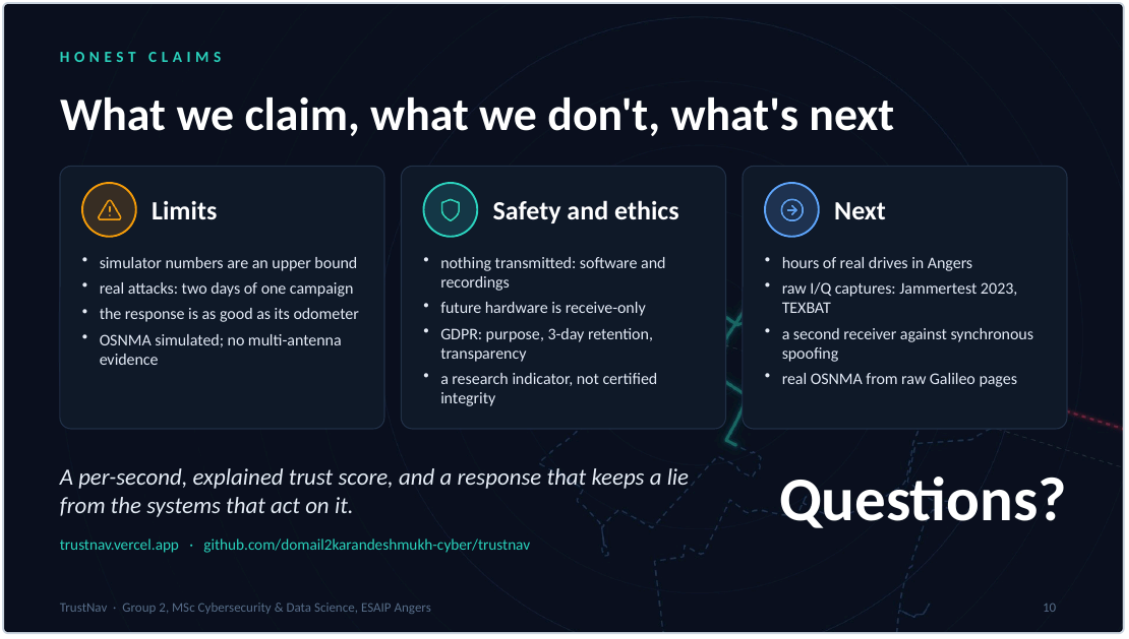
Two: the map. The reported track is coloured by trust and drawn hollow once refused; the protected position stays on the real path, with the dashed line showing the length of the lie. The response action reads ISOLATE, and the playbook lists what the dispatcher should do now.

Three: the phone. The page at /phone sends the phone's own fixes once a second; the laptop scores them like any receiver, and the phone appears on the live map with its trust score. The switch at the bottom overlays a teleport, a drift or jamming in software, nothing is transmitted, so the detection and the response can be shown on real streets. The phone gives no per-satellite signal strength, clock or OSNMA; that is what the receiver board adds later.

Under the hood: open source on GitHub, a Python engine with 42 automated tests, a Supabase back end with row-level security and a hashed ingest token, and a Next.js console with 109 end-to-end checks on every build.

If the network fails in the room, we have the replay recorded and the paper's figures; the demo degrades to the recorded scenarios.

10 What we claim, what we don't, what's next



Speaker: Pavan (1.5 min), then questions for the whole team

We want to be precise about what we claim. The detection table comes from one simulator, so those numbers are an upper bound. The real attacks we replayed are two days of one campaign, on two devices; that is evidence, not a statistic. The time rule needs a host clock, or a spoofer that moves time backwards; a spoofer that keeps time right must be caught by the other witnesses. The response layer is only as good as the odometer it trusts. OSNMA is simulated, and we have no multi-antenna or raw signal evidence.

On safety and ethics: we transmit nothing at any point. Our own attacks exist as software, the real ones are recordings made by others under licence, and the Phase 2 hardware is receive-only. A real deployment tracks drivers, so purpose limitation, retention, three days in the demo, access control and transparency apply under the GDPR. The trust score is a research indicator, not a certified integrity figure; TrustNav is not for aviation or maritime use.

What comes next: hours of real drives in Angers, first with the phone, then with a u-blox receiver, to get a false-alarm rate per hour on our own data; the raw radio recordings of Jammertest 2023 and TEXBAT through a software receiver; a second cheap receiver against synchronous spoofing; and real OSNMA from raw Galileo pages.

In one sentence: a per-second, explained trust score and a response that keeps a lie from the systems that act on it. Thank you, we are happy to take questions.

Likely questions and short answers

Is this only simulated data?

No. The detection table is simulated, because no public dataset has labelled per-second observables with routes and odometers and transmitting attacks is illegal. But we replayed public recordings of real attacks from Jammertest 2024 (the VAARA dataset, University of Vaasa): a phone captured by a real spoofer for 26.5 minutes, and a u-blox receiver driven past real jammers twenty times, plus a clean real drive. The first version missed the real spoofing for 21 min 49 s; with the sixteenth rule it is refused in the same second. One command reproduces it: python -m trustnav_engine.attackcheck.

What is new here compared with what exists?

Three things. One, the position: a software layer beside any receiver that scores every vehicle every second, explains the score in words with values, and responds by refusing the fix; receiver flags give a state code, probes protect a site, telematics displays what it is given. Two, the response layer with a rejoin rule a spoofer cannot defeat by looking clean. Three, the evidence: we publish where it failed on a real attack and what fixed it. Checking receiver time against another clock is not a new idea, timing receivers do it; what we add is showing on a public recording that it was the missing witness for a phone-class receiver, as an explainable rule with a measured before and after.

Why did the first version miss the real spoofing?

Because the spoofer was gentle. It moved the position 34 metres while the car was parked and then held it still: no jump, no impossible speed. The jump in signal strength lowered trust, but not under the alert line. The phone gives no pseudorange residuals, no clock data and no OSNMA, so those witnesses were absent. Only when the car drove away did the lie grow, and without an odometer the stream alone still looked like a parked car. Time was the one thing the spoofer had changed that we could check.

What does the time rule need, and can a spoofer defeat it?

It needs a second clock: the phone's or the gateway's system clock, kept by the network or a real-time clock, compared with the receiver's UTC with a five-second tolerance and a slowly adapting baseline. Without a host clock it still catches receiver time running backwards against its own one-per-second count. A spoofer that keeps time within five seconds of the truth is not caught by this rule; the route, the odometer, the signal checks and the model have to catch that one. On the clean data we have, the rule never fired: 0 alerts in 37 minutes before the capture and 9 minutes free of tests.

Isn't the model just learning the simulator?

Yes, and we show it: on the clean real drive it alerted on 89 % of the seconds until re-calibrated on that receiver, then on none in the remaining 8.5 minutes. At Jammertest we re-calibrated it on the 38 minutes recorded before the first test and it alerted on 15 of 20 jamming exposures. That is why it is trained on normal data only, never on attacks, and why re-calibration on the receiver in use is part of the deployment. The rules transfer as they are.

Why noisy-OR and not a classifier or an average?

Rules and model are two witnesses: either one should be able to raise the alarm. An average of 90 and 20 gives 55, barely over the line, and 90 with silent rules averages to 45, no alert; noisy-OR gives 75 and 72. A supervised classifier would learn the simulator's attacks. Noisy-OR assumes conditional independence, which jamming violates; the risk is then over-confident, which is harmless for an alert but means it should not be read as a probability. If asked to show it: Method page, "Try it", move the two sliders.

How did you choose the thresholds and weights?

Thresholds from physics and the platform profile, not from data. The two fusion weights by grid search on validation routes only; the threshold sweep in the paper shows F1 is flat around the chosen alert line of 50, so it is not tuned to the test set. The five-second time tolerance was chosen when we wrote the rule, after seeing the attack, but it is not fitted finely: the real jump was 62 seconds, then an hour, and on the clean data receiver time never left the tolerance.

What about false alarms in real traffic, tunnels, urban canyons?

Simulator, held-out: 0.44 % of clean seconds, four brief episodes in 37.5 minutes, no false incident. Real clean drive: zero from the rules in 27.6 minutes. Jammertest, outside the tests with a normal signal: zero from the rules on both days; the re-calibrated model 0.8 % on the jamming day and none on the spoofing morning. We still need hours of ordinary driving to state a rate per hour, and that is the first item of the plan.

A clever spoofer could follow the road at the right speed. Would you catch it?

A spoofer that matches the route, the speed, the signal spread, the clock and the time would be hard to detect from one receiver; that is in our limitations. The odometer comparison and the route are the first defence; a second antenna on the vehicle is the next one, because one spoofer makes two real antennas report the same position.

What does the odometer add, and what if it is unavailable?

It is an input the spoofer cannot touch: it feeds a speed-mismatch rule and the dead-reckoned estimate during a hold. Without it, the hold falls back to the last trusted speed and heading, and the estimate drifts faster. On a vehicle it comes from the CAN bus or OBD-II; on the phone demo we have no odometer. In the Jammertest replay the 45-metre figure uses the reference speed as the odometer, so it is a best case.

Why Galileo OSNMA and not GPS authentication?

OSNMA is the only operational civil authentication service, since 24 July 2025, and it is becoming mandatory in smart tachographs. GPS has no equivalent in service. OSNMA is one witness: it catches forged data, not a relay or jamming.

How is the system itself secured?

The browser only reads; all writes go through an edge function that checks a token stored as a SHA-256 hash; presenter controls use a bcrypt-hashed passcode with rate limiting; no secret is in the repository; row-level security on every table.

What is the false-positive cost of a hold?

A hold adds a few metres of odometer error even when the attack did not move the position (the signal-anomaly case, 20.8 m), and holds outside attacks total 26 seconds over the whole catalogue. The release rule and the ten-times bound are the trade-off between safety and availability.

Did the real data change anything else?

Yes, six defects in total, all fixed and tested. From the clean drive: tail scores that saturated on extreme values, and a sixty-second release that let a distant fix back in. From Jammertest: a sentence glued to a binary frame was dropped, a missing satellite count was guessed, a hold was stepped per observation and not per second, and the position-jump limit ignored gaps in the stream. None of them changed the simulator numbers.

What would you do with more time or budget?

About 250 euros buys a dual-band receiver board and antenna (a single-band board is about 60). The plan is: record normal drives, re-calibrate, run the raw radio captures through a software receiver, then a second receiver and real OSNMA. The code, adapters and calibration scripts for all of that already exist.

All numbers in these notes are those of the IEEE paper (trustnav.pdf, trustnav_fr.pdf) and are regenerated by paper/figures/make_figures.py, trustnav_engine.realcheck and trustnav_engine.attackcheck.